

1. 3D Trilateration with Four Anchors

This is the most basic way to find where a node is. The node measures its distance to four anchor nodes, and the anchors already know their own positions. From those four distances the node can work out its own x, y and z directly. There is no guessing and no repeating, so even a very weak node can finish it in a fraction of a millisecond.

Why four? If you know your distance to one anchor, you could be anywhere on a sphere around it. Two spheres cross in a circle. Three spheres cut that circle down to two points, and the fourth sphere picks the right one. Flat 2D work only needs three anchors. The height adds one more unknown, so 3D needs one more anchor.

p01_trilateration_3d.py

```
"""Find one node position in 3D using four anchor nodes."""
import numpy as np

# four anchors that already know their own (x, y, z) in metres
anchor_positions = np.array([[0, 0, 0], [50, 0, 5], [0, 50, 8],
                             [25, 25, 30]], dtype=float)
true_position = np.array([18.0, 22.0, 6.0]) # only used to check the answer
# distance from the node to each anchor (Pythagoras in 3D)
measured_distances = np.linalg.norm(anchor_positions - true_position, axis=1)
# subtract anchor 0's equation from the other three: the squares cancel
left_side = 2.0 * (anchor_positions[1:] - anchor_positions[0])
right_side = (measured_distances[0] ** 2 - measured_distances[1:] ** 2
              + np.sum(anchor_positions[1:] ** 2, axis=1)
              - np.sum(anchor_positions[0] ** 2))
estimated_position = np.linalg.solve(left_side, right_side) # 3 equations
print("true position      :", true_position)
print("estimated position:", np.round(estimated_position, 3))
print("error: %.4f m" % np.linalg.norm(estimated_position - true_position))
```

What it prints

```
true position      : [18. 22.  6.]
estimated position: [18. 22.  6.]
error: 0.0000 m
```

How the code works

The first few lines set up the example. `anchor_positions` holds the four anchors as rows of x, y, z in metres. `true_position` is where the node really is. In a real network nobody knows this, but here it is used to create the distances and to check the answer at the end. `measured_distances` is the straight line distance from the node to each anchor. `np.linalg.norm` does that in one call. It is just Pythagoras used twice: square the differences in x, y and z, add them up, take the square root.

Each distance gives one equation, but every one of them contains x squared, y squared and z squared, so they are not straight line equations and cannot be solved directly. The trick is to subtract the first anchor's equation from each of the other three. The squares cancel out, and what is left is three ordinary linear equations in three unknowns.

`left_side` and `right_side` are those three equations written as a 3 by 3 matrix and a vector of three numbers. `np.linalg.solve` then solves them in one line. The same two lines appear again in several of the later programs.

What the output means

The estimated position is (18, 22, 6), which is exactly the true position, and the error is 0.0000 m. That is expected, because the distances in this example are perfect. So this run is really a check that the setup and the maths are right. In a real network the distances would carry noise and the error would not be zero. Program 2 deals with that.

When to use it

Use it when a node can hear exactly four anchors and the distances are good. It is the cheapest way to get a position: one 3 by 3 solve. There is one thing to watch. If the four anchors sit almost in one flat plane, the equations cannot tell up from down, and the height comes out wrong even when the distances are perfect. Place some anchors high and some low.

2. Multilateration with Least Squares

Real distances always carry small mistakes. Signal strength changes when someone walks past, and clocks drift. So this program uses six anchors instead of four, and every distance has a small error added to it. With six noisy distances there is no single position that fits all of them exactly. Least squares picks the position that comes closest to fitting all six at once. The extra anchors cost nothing in hardware. The node simply listens to more of them, and their mistakes partly cancel each other out.

```

"""Use six noisy distances and let least squares find the best position."""
import numpy as np

anchor_positions = np.array([[0, 0, 0], [60, 0, 4], [0, 60, 6], [60, 60, 9],
                             [30, 30, 20], [10, 45, 12]], dtype=float)
true_position = np.array([31.0, 17.5, 7.2])
# fixed errors in metres, so the run gives the same result every time
distance_noise = np.array([0.5, -0.4, 0.6, -0.3, 0.4, -0.5])
measured_distances = (np.linalg.norm(anchor_positions - true_position, axis=1)
                      + distance_noise)

# same subtraction trick as program 1, now 5 equations for 3 unknowns
left_side = 2.0 * (anchor_positions[1:] - anchor_positions[0])
right_side = (measured_distances[0] ** 2 - measured_distances[1:] ** 2
              + np.sum(anchor_positions[1:] ** 2, axis=1)
              - np.sum(anchor_positions[0] ** 2))

# lstsq picks the position with the smallest total squared mismatch
estimated_position = np.linalg.lstsq(left_side, right_side, rcond=None)[0]
print("estimated position:", np.round(estimated_position, 2))
print("error: %.2f m" % np.linalg.norm(estimated_position - true_position))

```

What it prints

```

estimated position: [31.5  17.48  7.63]
error: 0.66 m

```

How the code works

The setup is the same as in program 1, with six anchors this time. `distance_noise` holds six fixed errors, all between 0.3 and 0.6 m in size, some positive and some negative. They are added to the perfect distances to make `measured_distances`. Fixed numbers are used instead of random ones so that the printed result is the same on every computer.

The equations are built exactly as before: subtract the first anchor's equation from the rest. With six anchors that leaves five equations for three unknowns. `np.linalg.solve` cannot handle that, because it needs exactly as many equations as unknowns. `np.linalg.lstsq` can. It returns the position for which the total of the squared mismatches is smallest. It is the same idea as drawing the best straight line through a cloud of dots. The `[0]` at the end picks the solution out of the several things that `lstsq` returns.

What the output means

The true position was (31.0, 17.5, 7.2). The estimate is (31.5, 17.48, 7.63), so the error is 0.66 m. That is about the same size as the noise that went in. Least squares did not remove the noise, but it stopped it from piling up. Extra anchors help because their random mistakes partly cancel each other.

When to use it

Use it almost always. If a node can hear more than four anchors, use every one of them. The cost is still one solve. One badly wrong reading can drag the answer, though, so readings that disagree strongly with the rest should be dropped before solving.

3. Turning Signal Strength into Distance

Programs 1 and 2 take the distances as given. This program shows the cheapest place to get them from: the strength of the radio signal that has already arrived. Every packet a node receives comes with a signal strength reading, called RSSI, so it costs nothing extra. A signal gets weaker the further it travels, so a weak signal means a far away anchor. The catch is that the reading is noisy, and this program shows how much that matters.

p03_rssi_pathloss_ranging.py

```
"""Turn radio signal strength into a distance in metres."""
import numpy as np

reference_power_dbm = -45.0      # signal strength measured at 1 metre
path_loss_exponent = 2.8        # how quickly the signal gets weaker
true_distances = np.array([10.0, 25.0, 40.0])
# fading formula: strength drops by 10 * exponent * log10(distance) dB
clean_power_dbm = (reference_power_dbm
                   - 10.0 * path_loss_exponent * np.log10(true_distances))
noisy_power_dbm = clean_power_dbm + np.array([2.0, -3.0, 1.5]) # dB noise
# the same formula turned around: strength back to metres
estimated_distances = 10.0 ** ((reference_power_dbm - noisy_power_dbm)
                               / (10.0 * path_loss_exponent))
print("true distances      :", true_distances)
print("estimated distances:", np.round(estimated_distances, 2))
print("distance errors    :",
      np.round(estimated_distances - true_distances, 2))
```

What it prints

```
true distances      : [10. 25. 40.]
estimated distances: [ 8.48 32.  35.36]
distance errors     : [-1.52  7.  -4.64]
```

How the code works

Two constants describe the radio link. `reference_power_dbm` is the signal strength at exactly one metre, here -45 dBm. `path_loss_exponent` says how quickly the signal fades with distance, here 2.8. The bigger this number, the faster the signal dies away.

The code then does a round trip. It starts from three true distances, 10, 25 and 40 m, and works out the signal strength each one would give using the standard fading formula: the

strength drops by 10 times the exponent times $\log_{10}$ of the distance, in decibels. That gives `clean_power_dbm`. Then it adds a few decibels of fixed noise to stand in for the real world. Finally it turns the formula around, using 10 to the power of something instead of $\log_{10}$, to get metres back out of the noisy strength. This last step is the only part a real node would run. The first part just makes the test data.

What the output means

The true distances were 10, 25 and 40 m. The estimates came out as 8.48, 32.00 and 35.36 m, so the errors are -1.5 m, +7.0 m and -4.6 m. A mistake of only 2 or 3 dB in the signal moved the answer by several metres. The formula works in ratios, so the same error in decibels means a bigger error in metres when the anchor is further away. This is why signal strength is only a rough guide to distance, and why it pays to average several packets before converting.

When to use it

Use it when the nodes have no special ranging hardware and a rough distance is good enough. The cost is almost nothing, one logarithm and one power, and no anchors are needed for this step. Two warnings. The error grows quickly with distance. And the two constants must be measured at the real site, because the ground and the plants around it change them. Values copied from a book will not match.

4. DV-Hop: Using Hop Counts Instead of Distance

Some networks cannot measure distance at all, because the radios are too simple.

DV-Hop gets around this by counting instead of measuring. Each anchor sends out a message that spreads through the network hop by hop, and every node counts how many hops it took to arrive. A node that is 3 hops from an anchor is roughly 3 times the average hop length away from it. Those rough distances then go into the same solve as program 1.

```

"""DV-Hop: guess distances from hop counts, then solve for the position."""
import numpy as np

anchor_positions = np.array([[0, 0, 0], [80, 0, 5], [0, 80, 6],
                             [80, 80, 20]], dtype=float)
true_position = np.array([30.0, 25.0, 8.0])
average_hop_distance = 15.0 # metres covered by one radio hop
# simulate the hop count: true distance / hop length, rounded to whole hops
hop_counts = np.round(np.linalg.norm(anchor_positions - true_position, axis=1)
                      / average_hop_distance)
guessed_distances = hop_counts * average_hop_distance # rough distances
# same solve as program 1, but fed with the rough distances
left_side = 2.0 * (anchor_positions[1:] - anchor_positions[0])
right_side = (guessed_distances[0] ** 2 - guessed_distances[1:] ** 2
              + np.sum(anchor_positions[1:] ** 2, axis=1)
              - np.sum(anchor_positions[0] ** 2))
estimated_position = np.linalg.solve(left_side, right_side)
# keep the answer inside the 80 x 80 x 20 m field
estimated_position = np.clip(estimated_position, 0.0, [80.0, 80.0, 20.0])
print("hop counts to anchors:", hop_counts)
print("estimated position   :", np.round(estimated_position, 2))
print("error: %.2f m" % np.linalg.norm(estimated_position - true_position))

```

What it prints

```

hop counts to anchors: [3. 4. 4. 5.]
estimated position   : [30.7 30.84 0. ]
error: 9.93 m

```

How the code works

average_hop_distance is set to 15 m, the distance one radio hop covers on average. In a real network the anchors work this value out and share it. Here the hop counts are simulated. The code takes the true distance to each anchor, divides by 15 and rounds to a whole number of hops, which gives 3, 4, 4 and 5. Multiplying back by 15 gives guessed_distances of 45, 60, 60 and 75 m. Compare those with the true distances of about 40, 56, 63 and 75 m and you can see the rounding error already built in.

From there it is program 1 again: build left_side and right_side, then np.linalg.solve. One extra line uses np.clip to keep the answer inside the field, between 0 and 80 m sideways and 0 and 20 m in height. Rough distances can push the solved position outside the area where the nodes actually are, and clipping stops that.

What the output means

The estimate is (30.7, 30.84, 0.0) against a true position of (30, 25, 8), so the error is 9.93 m. The x value is close and y is about 6 m off. The height shows the weak spot. The solve actually gave a height of about -6 m, below the ground, and the clip pulled it up to 0. The height suffers most with rough distances like these.

When to use it

Use it with cheap hardware in a dense network, where an error of about ten metres is acceptable. The maths on each node is light, but the flooding step costs radio messages across the whole network. Remember that the average hop length is only a guess, so the error stays near half the radio range.

5. Centroid and Weighted Centroid

This is the simplest idea of all ten. If a node can hear five anchors, it is probably somewhere in the middle of them. So take the average of their positions and call that the estimate. Then make it a bit smarter: anchors that are nearer should count for more. Give each anchor a weight and let the near ones pull harder. It is a handful of multiplications, so the weakest node in the network can run it.

p05_weighted_centroid_3d.py

```
"""Weighted centroid: average the anchors, but trust near anchors more."""
import numpy as np

anchor_positions = np.array([[0, 0, 2], [60, 0, 4], [0, 60, 3], [60, 60, 6],
                             [30, 30, 18]], dtype=float)
true_position = np.array([16.0, 38.0, 5.0])
measured_distances = np.linalg.norm(anchor_positions - true_position, axis=1)
weights = 1.0 / measured_distances      # near anchor = big weight
# weighted average: multiply each anchor by its weight, add up, divide
weighted_position = ((weights[:, None] * anchor_positions).sum(axis=0)
                     / weights.sum())
plain_position = anchor_positions.mean(axis=0)  # for comparison
print("plain average      :", np.round(plain_position, 2))
print("weighted average :", np.round(weighted_position, 2))
print("error plain=%.2f m   error weighted=%.2f m"
      % (np.linalg.norm(plain_position - true_position),
         np.linalg.norm(weighted_position - true_position)))
```

What it prints

```
plain average      : [30.  30.   6.6]
weighted average : [25.23 33.19  8.31]
error plain=16.20 m   error weighted=10.92 m
```

How the code works

`plain_position` is the plain average. `anchor_positions.mean(axis=0)` averages the x, y and z columns separately, which gives the centre of the five anchors.

For the weighted version, each anchor gets a weight of 1 divided by its distance. An anchor about 20 m away gets a weight of about 0.05, one about 60 m away gets about 0.017, so the near one counts nearly three times as much. `weights[:, None]` reshapes the list of weights so that each weight multiplies its own anchor row. The weighted rows are added up and divided by the total weight, which is exactly what a weighted average is. The distances here are perfect, but in practice a rough distance or even the raw signal strength is enough to build the weights.

What the output means

The true position is (16, 38, 5). The plain average lands at (30, 30, 6.6), which is just the middle of the field, 16.20 m away. The weighted average moves towards the nearer anchors and lands at (25.23, 33.19, 8.31), 10.92 m away. So the weighting cut the error by about a third for almost no extra work. The answer is still rough, because no equations are solved. Think of it as a first guess rather than a final answer.

When to use it

Use it on very low power nodes, or as the starting guess for program 7. It needs four or more anchors within hearing range and no distance hardware at all. One built in limit: the answer can never land outside the area covered by the anchors, so a node near the edge of the field is always pulled inwards.

6. The APIT Test: Am I Inside These Four Anchors?

This program measures no distance at all. It asks one yes or no question: is the node inside the shape formed by four anchors? Four points in 3D make a tetrahedron, a pyramid with a triangle as its base. If the node is inside it, then the middle of that shape is a fair estimate. A full APIT system tests many groups of four anchors and keeps only the region where all the inside answers overlap, which narrows the estimate down. This program shows the test itself, for one group.

p06_apit_tetrahedron_3d.py

```
"""APIT idea: check if the node sits inside the shape made by 4 anchors."""
import numpy as np

anchor_positions = np.array([[0, 0, 0], [50, 0, 4], [0, 50, 6],
                             [20, 20, 25]], dtype=float)
true_position = np.array([15.5, 13.0, 5.95])
first_anchor = anchor_positions[0]
# three edge directions from the first corner to the other three corners
corner_directions = np.column_stack([anchor_positions[1] - first_anchor,
                                      anchor_positions[2] - first_anchor,
                                      anchor_positions[3] - first_anchor])
# how much of each edge direction is needed to reach the node
corner_shares = np.linalg.solve(corner_directions,
                                true_position - first_anchor)
# inside when no share is negative and the shares add up to 1 or less
is_inside = corner_shares.min() >= 0.0 and corner_shares.sum() <= 1.0
estimated_position = anchor_positions.mean(axis=0) # centre of the shape
print("corner shares:", np.round(corner_shares, 2), " inside:", is_inside)
print("estimated position:", np.round(estimated_position, 2))
print("error: %.2f m" % np.linalg.norm(estimated_position - true_position))
```

What it prints

```
corner shares: [0.25 0.2  0.15]  inside: True
estimated position: [17.5  17.5   8.75]
error: 5.66 m
```

How the code works

The test uses a small trick. Stand at the first anchor and look at the other three. They give three directions, stored as the columns of `corner_directions`. Any point can be written as the first anchor plus some share of each of those three directions. `np.linalg.solve` finds the three shares for the node, and they end up in `corner_shares`.

The shares say where the node is relative to the corners. If any share is negative, the node is on the wrong side of a face. If the shares add up to more than 1, the node is past the far face. So the node is inside exactly when every share is 0 or more and their total is 1 or less. That is the `is_inside` line. The estimate is then simply the average of the four

anchors, the centre of the shape. Note that this demo runs the test on `true_position`, so you can see how the geometry works. A real node does not know its true position. It has to decide inside or outside from what it can hear.

What the output means

The shares are 0.25, 0.2 and 0.15. None is negative and they add up to 0.6, so `inside` is `True`. The centre of the shape is (17.5, 17.5, 8.75), the true position is (15.5, 13.0, 5.95), and the error is 5.66 m. With only one group of anchors the answer is coarse. The accuracy in a full APIT system comes from overlapping many groups.

When to use it

Use it in range free networks with many anchors, so that many groups can be tested and their overlap taken. Each test is one small solve, so the cost per group is low. It needs four or more anchors and no distance measurements.

7. Step by Step Correction (Maximum Likelihood)

Instead of solving for the position in one go, this program starts from a rough guess and improves it a few times. Each round it asks: if I really were at my current guess, how far would I be from each anchor, how does that compare with what I measured, and which small move would close the gap best? Then it makes that move. Five rounds are enough here. Of all ten programs, this one gives the best accuracy for the least extra work.

`p07_mle_gauss_newton_3d.py`

```
"""Start from a rough guess and correct it five times to fit distances."""
import numpy as np

anchor_positions = np.array([[0, 0, 0], [70, 0, 5], [0, 70, 8], [70, 70, 15],
                             [35, 35, 25]], dtype=float)
true_position = np.array([44.0, 21.0, 11.0])
measured_distances = (np.linalg.norm(anchor_positions - true_position, axis=1)
                      + np.array([0.6, -0.5, 0.4, -0.6, 0.5])) # noise
estimated_position = anchor_positions.mean(axis=0) # rough starting guess
for step in range(5):
    offsets = estimated_position - anchor_positions
    guessed_distances = np.linalg.norm(offsets, axis=1) # from the guess
    directions = offsets / guessed_distances[:, None] # unit vectors
    # find the small move that best closes the distance gaps
    correction = np.linalg.lstsq(directions,
                                  measured_distances - guessed_distances,
                                  rcond=None)[0]
    estimated_position = estimated_position + correction
print("estimated position:", np.round(estimated_position, 2))
print("error: %.2f m" % np.linalg.norm(estimated_position - true_position))
```

What it prints

```
estimated position: [44.77 21.23 10.51]
error: 0.95 m
```

How the code works

Five anchors get noisy distances, with fixed errors of about half a metre, just like program 2. The starting guess is the plain average of the anchor positions, the same idea as program 5. That guess is about 17 m from the truth.

Inside the loop, `offsets` is the vector from each anchor to the current guess, and `guessed_distances` is the length of each of those, so the distance the guess would have from each anchor. `directions` divides each offset by its length, giving a direction of length 1 for each anchor. The `lstsq` call is the heart of it. Moving the guess by a small step changes the distance to an anchor by roughly the part of that step that points along the anchor's direction. So `lstsq` looks for the step that best closes all five gaps between `measured_distances` and `guessed_distances` at once. That step is correction, and it is added to the guess. Each round the gaps shrink, so the corrections shrink too.

What the output means

After five rounds the estimate is (44.77, 21.23, 10.51) against a true position of (44, 21, 11), an error of 0.95 m. The guess started about 17 m off and ended under a metre off. That remaining metre is about the size of the noise that was added to the distances.

When to use it

Use it after any cheap first guess, whenever distances are available. This is the practical choice for real work. The cost is moderate, five small solves, and it needs four or more anchors with distances. It does need a sensible starting guess. With badly placed anchors it can settle on a wrong answer.

8. MDS-MAP: Building a Map from a Distance Table

All the programs so far find one node at a time. MDS-MAP does the whole network at once. Give it the table of distances between every pair of nodes, and one matrix calculation rebuilds the shape of the entire network. The shape comes out exact, but it floats freely. It is not yet placed on the real map, and it could be turned or flipped. A few anchors are used at the end to turn and shift it into place. This program shows the rebuilding step and checks that the shape is right.

p08_mds_map_3d.py

```
"""MDS-MAP: build a relative map of all nodes from distances only."""
import numpy as np

node_positions = np.array([[0, 0, 0], [40, 0, 5], [0, 40, 8], [40, 40, 12],
                           [20, 20, 20]], dtype=float)
# 5 x 5 table: the distance between every pair of nodes
distance_table = np.linalg.norm(node_positions[:, None, :]
                                - node_positions[None, :, :], axis=2)
node_count = len(node_positions)
all_ones = np.ones((node_count, node_count))
centering = np.eye(node_count) - all_ones / node_count
# classical MDS: squared distances -> table of inner products
inner_products = -0.5 * centering.dot(distance_table ** 2).dot(centering)
eigenvalues, eigenvectors = np.linalg.eigh(inner_products)
largest_three = np.argsort(eigenvalues)[::-1][:3] # the 3 dimensions
relative_map = (eigenvectors[:, largest_three]
                * np.sqrt(eigenvalues[largest_three]))
# check: do the distances in the new map match the original table?
rebuilt_table = np.linalg.norm(relative_map[:, None, :]
                                - relative_map[None, :, :], axis=2)
print("relative map of node 5:", np.round(relative_map[4], 2))
print("largest distance mistake in the map: %.6f m"
      % np.abs(rebuilt_table - distance_table).max())
```

What it prints

```
relative map of node 5: [ 2.57 -0. -10.7 ]
largest distance mistake in the map: 0.000000 m
```

How the code works

`node_positions` holds five nodes, and `distance_table` is the 5 by 5 table of distances between them. It is built by subtracting every position from every other position and taking the lengths. In a real network, pairs of nodes that cannot hear each other would have their entry filled in with the shortest path distance through other nodes.

The next lines are the classical MDS recipe. `centering` is a matrix that removes the average from whatever it multiplies. Squaring the distances and applying centering from both sides turns the distance table into `inner_products`, a table that describes the

positions relative to their common centre. `np.linalg.eigh` breaks that table into eigenvalues and eigenvectors. The three largest eigenvalues carry the three dimensions of the map, and scaling their eigenvectors by the square root of each eigenvalue gives `relative_map`, one row of x, y, z per node. The last step is a check. `rebuilt_table` works out all the distances from the new map and compares them with the original table.

What the output means

The relative map puts node 5 at (2.57, -0.0, -10.7). That looks nothing like its true position of (20, 20, 20), and that is expected. The map has the right shape but its own centre and its own orientation. The proof is the second line: the largest difference between any rebuilt distance and the original one is 0.000000 m. Every distance was recovered exactly, so the shape is right. Turning and shifting it onto the anchors would then give the real positions.

When to use it

Use it at the base station, for a whole network at once, when very few anchors are available. Four or more anchors are needed, but only to align the finished map. It cannot run on a node. It needs the full distance table in memory, and the matrix work grows with the cube of the node count.

9. Particle Swarm Search

This program finds the position by searching for it. It drops twenty random guesses, called particles, into the field. Each particle is scored by how badly its distances to the anchors disagree with the measured distances. Then every particle moves a little towards the best guess found so far. After sixty rounds the swarm has gathered around the answer. No calculus and no equation solving are needed, so it still works when the error formula is awkward or has many dips.

```

"""Particle swarm: many guesses fly towards the best guess so far."""
import numpy as np

anchor_positions = np.array([[0, 0, 0], [60, 0, 5], [0, 60, 6],
                             [60, 60, 20]], dtype=float)
true_position = np.array([37.0, 12.0, 8.5])
measured_distances = (np.linalg.norm(anchor_positions - true_position, axis=1)
                      + np.array([0.4, -0.3, 0.5, -0.4])) # noise
random_maker = np.random.default_rng(1) # fixed seed, repeatable run
# 20 random guesses spread over the 60 x 60 x 20 m field, standing still
particle_positions = random_maker.uniform([0, 0, 0], [60, 60, 20], (20, 3))
particle_speeds = np.zeros((20, 3))
for step in range(60):
    # 20 x 4 table: distance from every particle to every anchor
    distances = np.linalg.norm(particle_positions[:, None, :]
                              - anchor_positions[None, :, :], axis=2)
    # score each particle: low means its distances fit the measurements
    particle_errors = ((distances - measured_distances) ** 2).sum(axis=1)
    best_position = particle_positions[particle_errors.argmin()]
    # keep 70% of the old speed, add a random push towards the best guess
    particle_speeds = (0.7 * particle_speeds + 1.4
                      * random_maker.random((20, 3))
                      * (best_position - particle_positions))
    particle_positions = particle_positions + particle_speeds
print("best position found:", np.round(best_position, 2))
print("error: %.2f m" % np.linalg.norm(best_position - true_position))

```

What it prints

```

best position found: [37.43 11.9  9.29]
error: 0.90 m

```

How the code works

Four anchors get noisy distances, as before. `random_maker` is a random number generator with a fixed seed of 1, so the random starting guesses are the same on every run and the output is repeatable. `particle_positions` is 20 rows of x, y, z spread evenly over the 60 by 60 by 20 m field. `particle_speeds` starts at zero.

Each round, `distances` is a 20 by 4 table holding the distance from every particle to every anchor. `particle_errors` squares the gap between those and the measured distances and adds up the four gaps for each particle, so a low score means a good guess. `best_position` is the particle with the lowest score. The speed update has two parts. Keep 70 percent of the old speed, so particles carry some momentum, and add a random push, up to 1.4 times the gap, towards the best position. Then every particle moves by its speed. Over sixty rounds the particles close in on the best fit.

What the output means

The best particle ends at (37.43, 11.9, 9.29) against a true position of (37, 12, 8.5), an error of 0.90 m. That is about the same as program 7 reached with five small corrections. The difference is the cost. This program computed 20 particles times 60 rounds, so thousands of distance calculations, to get there.

When to use it

Use it when a direct solve keeps failing, or when the measurement model is unusual. Run it at the base station. It performs thousands of calculations for one node, which is fine on a computer and wasteful on a battery.

10. Kalman Filter for a Moving Node

A moving node gets a new position fix at every time step, and each fix is noisy, so the raw track jumps about. A Kalman filter smooths it. The idea is simple: do not throw away what you knew a moment ago. Mix the new fix with the previous estimate, and trust each one according to how sure you are of it. It sends no extra radio messages, which makes the improvement almost free.

p10_kalman_tracking_3d.py

```
"""Kalman filter: smooth the noisy position fixes of a moving node."""
import numpy as np

# the node moves 1 m in x per step; these are its true positions
true_track = np.array([[10, 5, 2.0], [11, 5.5, 2.1], [12, 6, 2.2],
                       [13, 6.5, 2.3], [14, 7, 2.4]])
# the noisy position fixes it actually receives, each nearly 3 m off
noisy_fixes = true_track + np.array([[2.0, -1.5, 0.8], [-1.8, 2.0, -0.7],
                                     [1.5, -2.0, 0.6], [-2.2, 1.7, -0.9],
                                     [2.1, -1.6, 0.7]])
estimated_position, uncertainty, measurement_noise = noisy_fixes[0], 4.0, 4.0
smoothed_track = np.zeros_like(noisy_fixes)
for step, one_fix in enumerate(noisy_fixes):
    # how much to trust the new fix (0 = ignore it, 1 = take it fully)
    kalman_gain = uncertainty / (uncertainty + measurement_noise)
    # move part of the way from the old estimate towards the new fix
    estimated_position = (estimated_position
                          + kalman_gain * (one_fix - estimated_position))
    # more sure after the update, then a bit less sure as the node moves on
    uncertainty = (1.0 - kalman_gain) * uncertainty + 0.5
    smoothed_track[step] = estimated_position
print("true / noisy / smoothed at the last step:")
print(np.round([true_track[-1], noisy_fixes[-1], smoothed_track[-1]], 2))
print("error of noisy fixes=%.2f m after filtering=%.2f m"
      % (np.sqrt(((noisy_fixes - true_track) ** 2).sum(1).mean()),
         np.sqrt(((smoothed_track - true_track) ** 2).sum(1).mean())))
```

What it prints

```
true / noisy / smoothed at the last step:
[[14.    7.    2.4 ]
 [16.1   5.4   3.1 ]
 [12.89  5.68  2.42]]
error of noisy fixes=2.73 m   after filtering=1.72 m
```

How the code works

`true_track` is five positions of a node moving in a straight line, one metre in *x* per step and half a metre in *y*. `noisy_fixes` is that track with fixed errors added, so that each fix lands nearly 3 m from the truth. Those fixes stand for the output of any of the earlier programs, run once per time step. The estimate starts out as the very first fix, since there is nothing better yet. The filter keeps two numbers. `uncertainty` says how unsure it is about its current estimate, and `measurement_noise` says how unsure it is about any new fix. Both start at 4.0.

Each step, `kalman_gain` is the share of trust to give the new fix: the estimate's uncertainty divided by the total of both uncertainties. With both at 4.0 the first gain is 0.5, so the first update lands halfway between the old estimate and the new fix. The estimate then moves that share of the way towards the fix. After the update the filter is more sure than before, so uncertainty is shrunk by (1 minus gain), and then 0.5 is added back because the node has moved on since the last fix. By the last step the gain has settled near 0.3, so each new fix moves the estimate about a third of the way. Every smoothed position is stored in `smoothed_track`.

What the output means

At the last step the true position is (14, 7, 2.4), the raw fix says (16.1, 5.4, 3.1), and the smoothed estimate says (12.89, 5.68, 2.42). The last line gives the RMSE over all five steps. RMSE means: square every error, take the average, then take the square root, so large errors count more. The raw fixes were 2.73 m off and the filtered track 1.72 m off. Notice that the smoothed *x* of 12.89 is behind the true 14. This simple filter has no speed term, so it lags a little behind a moving node. Adding speed to the model would fix that.

When to use it

Use it for any node that moves, and for any node whose fixes jump around from one reading to the next. It costs a few lines of maths per new fix, needs no extra anchors, and sends no extra messages.